

ONLINE SHOPPERS PURCHASING INTENTION PREDICTION

1. Project Overview

This project aims to predict whether an online shopper will complete a purchase based on their browsing behavior. Various machine learning classification algorithms were implemented and compared to identify the most effective model. After evaluating their performance, **a tuned XGBoost classifier** was selected as the final model due to its high accuracy and strong predictive capability. The trained model was deployed locally using FastAPI to provide real-time predictions through a REST API.

2. Problem Statement

Online retailers receive thousands of website visitors every day, but only a small percentage make a purchase. Identifying potential buyers based on their browsing behavior helps businesses improve marketing strategies, personalize recommendations, and increase conversion rates. The objective is to build a machine learning model that accurately predicts whether a visitor will make a purchase.

3. Objectives

The objectives of this project are:

- To analyze the online shoppers' browsing dataset.
- To preprocess the data for machine learning.
- To compare multiple classification algorithms.
- To optimize the best-performing model using hyperparameter tuning.
- To evaluate the model using appropriate performance metrics.
- To deploy the trained model using FastAPI for real-time prediction.

4. Dataset Description:

The Online Shoppers Purchasing Intention dataset contains information about users' browsing sessions on an e-commerce website. It includes numerical and categorical features describing visitor behavior, such as page visits, browsing duration, bounce rates, visitor type, and weekend visits. The target variable, **Revenue**, indicates whether the visitor completed a purchase.

ROWS: 12,330

FEATURES: 17 COLUMNS

TARGET VARIABLE: REVENUE(PURCHASE OR NO PURCHASE)

5. Exploratory Data Analysis

Data Preprocessing

The following preprocessing steps were carried out to prepare the dataset for machine learning:

1. **Imported Required Libraries:** Imported essential Python libraries such as Pandas, NumPy, Matplotlib, Seaborn, and Scikit-learn for data manipulation, visualization, preprocessing, and model development.
2. **Loaded the Dataset:** The Online Shoppers Purchasing Intention dataset was loaded into a Pandas DataFrame for analysis.
3. **Head() Function:** Used the head() function to display the first five records of the dataset and verify that the data was loaded correctly.
4. **Tail() Function:** Used the tail() function to display the last five records of the dataset and inspect the end of the dataset.
5. **Dataset Information:** Used the info() function to examine the dataset structure, including the number of rows and columns, data types of each feature, and to check for missing values.

The screenshot shows a Jupyter Notebook interface with a file explorer on the left and a code cell in the center. The file explorer lists files like `app.py`, `best_xgb_model.pkl`, `Main Project file.ipynb`, `online_shoppers_intention.csv`, `read.md`, `requirements.txt`, and `scaler.pkl`. The code cell displays the output of a `pandas.DataFrame` inspection, showing 12330 entries and 18 columns. The columns and their data types are listed below.

#	Column	Non-Null Count	Dtype
0	Administrative	12330 non-null	int64
1	Administrative_Duration	12330 non-null	float64
2	Informational	12330 non-null	int64
3	Informational_Duration	12330 non-null	float64
4	ProductRelated	12330 non-null	int64
5	ProductRelated_Duration	12330 non-null	float64
6	BounceRates	12330 non-null	float64
7	ExitRates	12330 non-null	float64
8	PageValues	12330 non-null	float64
9	SpecialDay	12330 non-null	float64
10	Month	12330 non-null	str
11	OperatingSystems	12330 non-null	int64
12	Browser	12330 non-null	int64
13	Region	12330 non-null	int64
14	TrafficType	12330 non-null	int64
15	VisitorType	12330 non-null	str
16	Weekend	12330 non-null	bool
17	Revenue	12330 non-null	bool

The output also includes `dtypes: bool(2), float64(7), int64(7), str(2)` and `memory usage: 1.5 MB`.

6. **Missing Value Check:** No missing values were found in the dataset.

The screenshot shows a Jupyter Notebook interface with a file explorer on the left and a code cell in the center. The file explorer lists files like `app.py`, `best_xgb_model.pkl`, `Main Project file.ipynb`, `online_shoppers_intention.csv`, `read.md`, `requirements.txt`, and `scaler.pkl`. The code cell displays the output of a `df.isnull().sum().sum()` operation, which returns `np.int64(0)`. Below the code cell, there is a section titled "4.1 Missing Value Treatment" with an observation stating that the dataset contains 0 missing values.

4.1 Missing Value Treatment

Observation:

- The missing value analysis confirms that the dataset contains **0 missing values**, indicating complete data availability across all features.
- No feature requires **missing value treatment or imputation** before further analysis.
- The absence of missing values improves data quality and ensures a smoother **data preprocessing workflow**.
- The dataset is ready for subsequent steps, including **feature engineering, model training, and performance evaluation**.

4.2 Duplicate Handling

7. **Duplicate Records:** A total of **125 duplicate records** were identified and removed from the dataset.

The image displays two sequential screenshots of a Jupyter Notebook environment within a Visual Studio Code editor. The notebook is titled 'Mini_Project' and is running on Python 3.14.4. The left sidebar shows the Explorer view with the file structure of the project, including files like 'app.py', 'best_xgb_model.pkl', 'Main Project file.ipynb', 'online_shoppers_intention.csv', 'read.md', 'requirements.txt', and 'scaler.pkl'. The main editor area shows the notebook content.

Top Screenshot: The notebook is at a section titled '4.2 Duplicate Handling'. It contains a code cell with the following Python code:

```
# Check the number of duplicate rows
df.duplicated().sum()

np.int64(125)
```

The output of the code is 125, indicating the number of duplicate records. Below the code cell, there is an 'Observation' section with the following text:

✓ **Observation:**

- A total of **125 duplicate records** were identified in the dataset during the data quality assessment.
- These duplicate entries may introduce **redundancy and bias** in the analysis and machine learning model performance.
- The identified duplicate rows will be **removed during preprocessing** to maintain data consistency and improve dataset quality.
- Cleaning duplicate records ensures that the model learns from **unique and meaningful data patterns**.

Bottom Screenshot: The notebook is at a section titled 'Cleaning duplicate records'. It contains a code cell with the following Python code:

```
# Remove duplicate rows
df = df.drop_duplicates()

# Verify duplicates are removed
df.duplicated().sum()

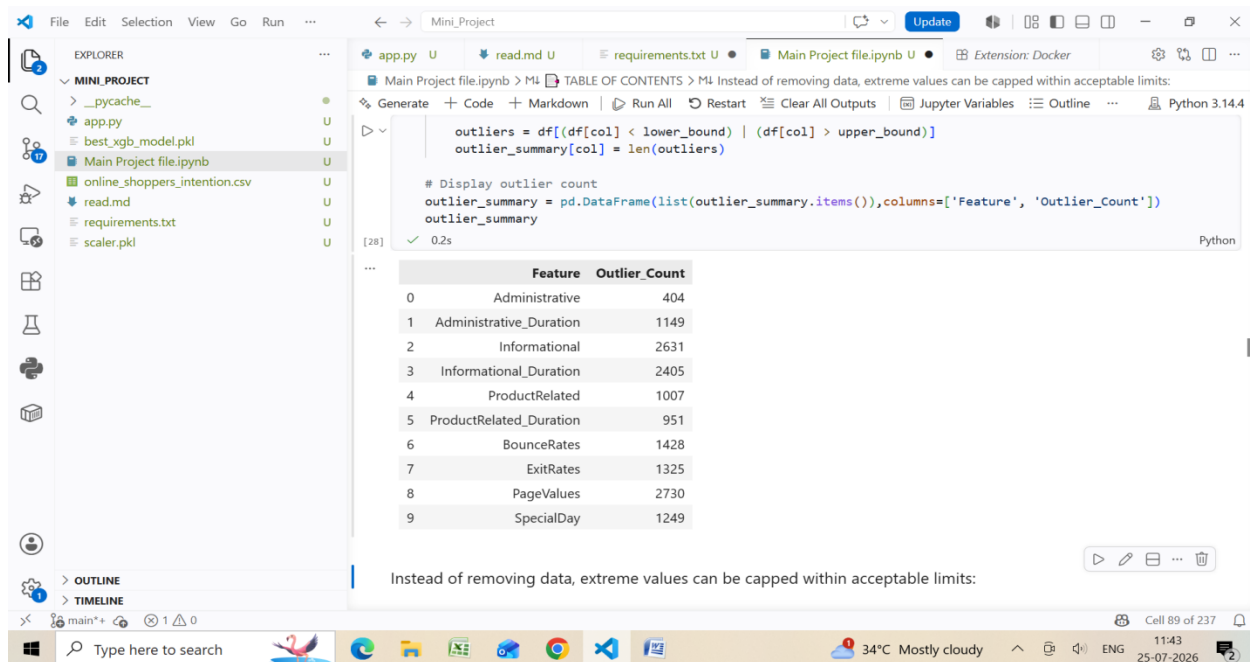
np.int64(0)
```

The output of the code is 0, indicating that all duplicate records have been removed. Below the code cell, there is an 'Observation' section with the following text:

✓ **Observation:**

- Initially, the dataset contained **125 duplicate records**, which were identified during the data cleaning process.
- The duplicate entries were removed using the `drop_duplicates()` method to eliminate redundant information.
- After the removal process, the dataset contains **0 duplicate records**, confirming successful data cleaning.
- The cleaned dataset ensures improved **data consistency, reliability, and accuracy** for further analysis and machine learning model development.

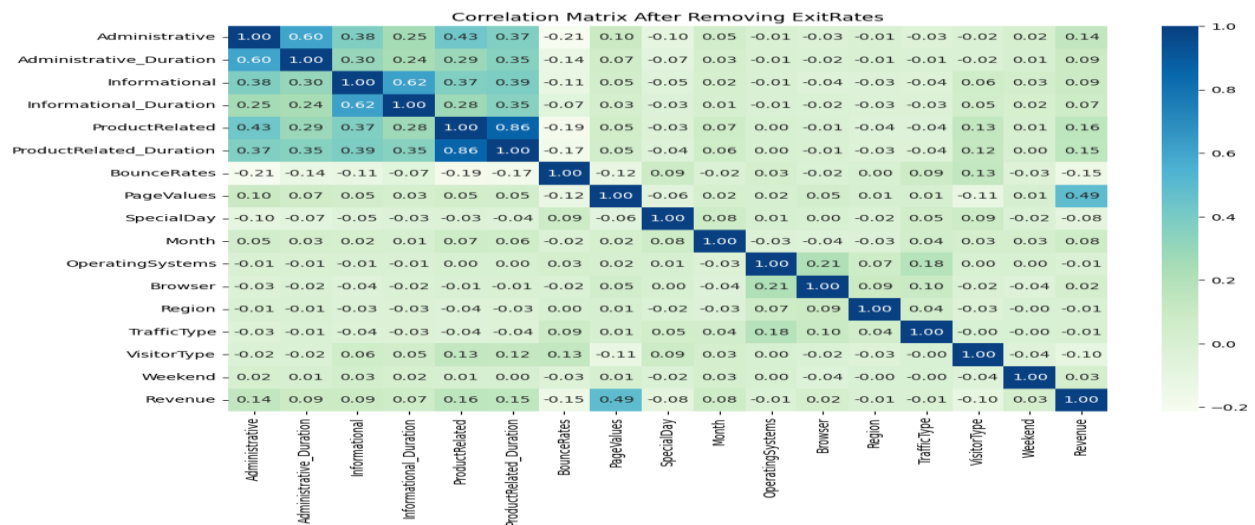
8. **Outlier Detection:** Outliers were detected in the numerical features using the Interquartile Range (IQR) method.



9. **Outlier Treatment:** The detected outliers were handled by limiting (capping) the extreme values within the acceptable range instead of removing the records.
10. **Correlation analysis** was performed to identify the relationship between numerical features and to detect highly correlated variables. A correlation matrix and **heatmap** were used to visualize the strength and direction of correlations between features.

Observations:

- **BounceRates** and **ExitRates** showed a **very high positive correlation (0.91)**, indicating that both features contain similar information. Therefore, **ExitRates** was removed to reduce multicollinearity.
- **ProductRelated** and **ProductRelated_Duration** exhibited a **strong positive correlation (0.86)**, showing that users who visit more product pages tend to spend more time on them. Both features were retained because they capture different aspects of browsing behavior.
- **PageValues** had the **highest positive correlation (0.49)** with the target variable (**Revenue**), making it the most influential feature for predicting purchase intention.
- Most other features showed **weak to moderate correlations**, indicating that they contribute unique information to the prediction model.



11. Categorical Encoding: Categorical variables (**Month**, **VisitorType**, and **Weekend**) were converted into numerical values using Label Encoding.

```

# Check data types
print(df_encoded.dtypes)
df=df_encoded

[36]: 0.4s Python

Administrative      int64
Administrative_Duration  float64
Informational      int64
Informational_Duration  float64
ProductRelated     int64
ProductRelated_Duration  float64
BounceRates        float64
ExitRates          float64
PageValues         float64
SpecialDay         float64
Month              int64
OperatingSystems   int64
Browser            int64
Region             int64
TrafficType        int64
VisitorType        int64
Weekend            int64
Revenue            int64
dtype: object

```

12. Feature Scaling: Numerical features were standardized using **StandardScaler** to ensure that all features had a similar scale before model training.

13. Feature Engineering: Two new features, **Total_Page_Visits** and **Total_Duration**, were created by combining related page count and duration features.

14. **Feature Selection:** The **SelectKBest** method was applied to identify the most significant features contributing to purchase prediction.

Final Features: ['Administrative', 'Informational', 'ProductRelated', 'ProductRelated_Duration', 'BounceRates', 'PageValues', 'VisitorType', 'Total_Duration', 'Total_Page_Visits']

15. **Train-Test Split:** The preprocessed dataset was divided into **80% training data** and **20% testing data** for model training and evaluation.

6. Handling Imbalanced Data using SMOTE

Introduction

The target variable (**Revenue**) was imbalanced because the majority of website visitors did not make a purchase. Machine learning models trained on imbalanced data tend to favor the majority class, leading to poor prediction of purchasing customers.

To overcome this issue, the **Synthetic Minority Oversampling Technique (SMOTE)** was applied.

Class Distribution Before SMOTE

Revenue	Count
No Purchase (0)	8238
Purchase (1)	1526

Class Distribution After SMOTE

Revenue	Count
No Purchase (0)	8238
Purchase (1)	8238

7. Model Training

After preprocessing, feature selection, and balancing the training data using SMOTE, multiple machine learning algorithms were trained to predict customer purchase intention.

Model	Accuracy (Before)	Accuracy (After)	Recall (Class 1)	F1-score (Class 1)	True Positives (TP)	False Negatives (FN)	Overall Analysis
Logistic Regression	88.69%	87.30%	0.39 → 0.76	0.52 → 0.65	149 → 290	233 → 92	Accuracy decreased slightly, but recall and F1-score improved significantly, resulting in better detection of the minority class.
Decision Tree	87.10%	85.13%	0.60 → 0.63	0.59 → 0.57	229 → 240	153 → 142	SMOTE slightly improved recall and minority class detection, but accuracy and F1-score decreased marginally.
Random Forest	90.70%	88.57%	0.61 → 0.74	0.67 → 0.67	232 → 282	150 → 100	Improved recall and reduced false negatives while maintaining the same F1-score. Overall performance remained strong after SMOTE.
Support Vector Machine (SVM)	89.06%	86.64%	0.46 → 0.72	0.57 → 0.63	177 → 276	205 → 106	Significant improvement in recall and F1-score with many more minority class instances correctly identified.
Boosting Model	90.21%	89.76%	0.61 → 0.69	0.66 → 0.68	234 → 264	148 → 118	Very small reduction in accuracy with improved recall and F1-score, resulting in balanced classification performance.

Random Forest and **XGBoost** were selected for hyperparameter tuning because they achieved the highest overall performance among the evaluated models after applying SMOTE.

8. Hyperparameter Tuning

Hyperparameters are settings that control the learning process of a machine learning model. Choosing appropriate hyperparameter values significantly improves model performance.

In this project, **RandomizedSearchCV** was used to optimize the XGBoost classifier and Random Forest.

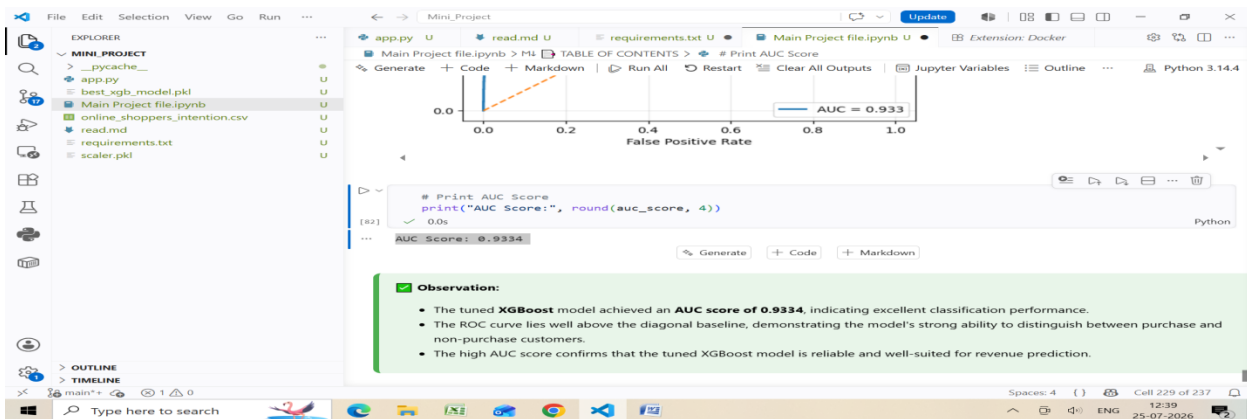
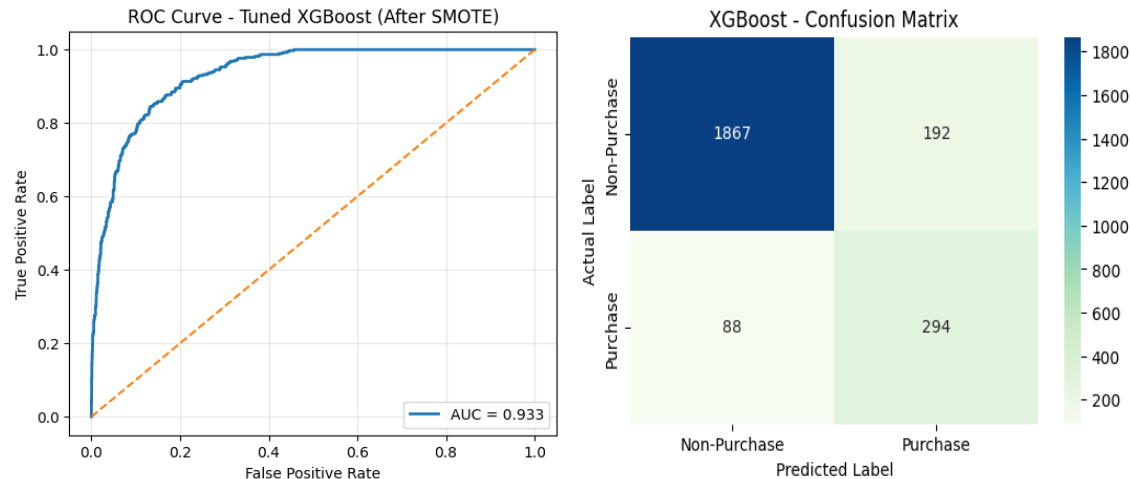
COMPARISON OF TUNED RANDOM FOREST AND TUNED XGBOOST

METRIC	TUNED RANDOM FOREST	TUNED XGBOOST	BETTER MODEL
 Accuracy	88.45%	88.53%	 Tuned XGBoost
 Precision (Class 0)	0.95	0.95	 Equal
 Recall (Class 0)	0.91	0.91	 Equal
 F1-score (Class 0)	0.93	0.93	 Equal
 Precision (Class 1)	0.61	0.60	 Tuned Random Forest
 Recall (Class 1)	0.74	0.77	 Tuned XGBoost
 F1-score (Class 1)	0.67	0.68	 Tuned XGBoost
 True Positives (TP)	281	294	 Tuned XGBoost
 False Negatives (FN) (Lower is better)	101	88	 Tuned XGBoost
 False Positives (FP) (Lower is better)	181	192	 Tuned Random Forest
 True Negatives (TN)	1878	1867	 Tuned Random Forest
 Overall Performance	Good overall performance with higher precision for the minority class and fewer false positives.	Higher accuracy, better recall, higher F1-score, more true positives and fewer false negatives.	 Tuned XGBoost is the better model overall and selected as the final model for deployment.

9. Final Model :

Reasons for Selecting Tuned XGBoost

- Highest overall accuracy among the evaluated models.
- Excellent **AUC score (0.9334)**, indicating strong class discrimination.
- Balanced Precision and Recall.
- Reduced overfitting through hyperparameter tuning.
- High robustness and scalability.



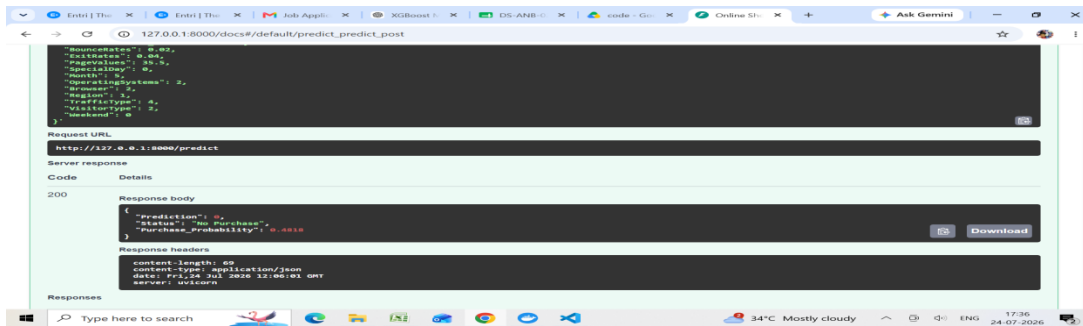
10. Deployment FastAPI

After selecting the tuned XGBoost classifier, the trained model was deployed using **FastAPI**, a modern Python web framework for building high-performance REST APIs.

API Development

A FastAPI application (app.py) was created to:

- Loaded the trained XGBoost model.
- Loaded the saved StandardScaler.
- Accepted user input through the /predict endpoint.
- Preprocessed the input data.
- Generated predictions using the trained model.
- Returned the prediction result and purchase probability in JSON format.

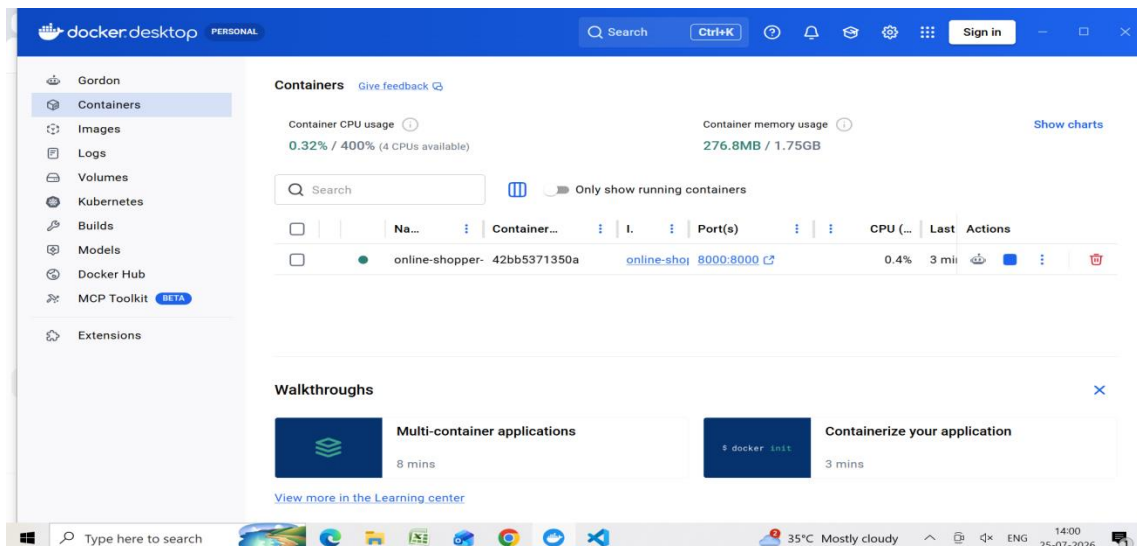


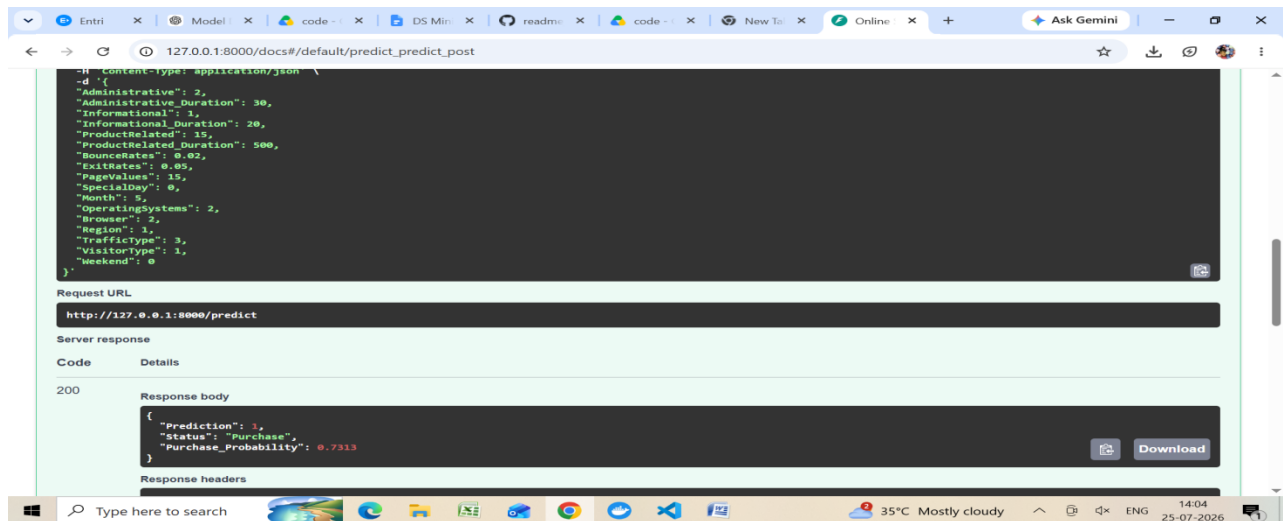
11. Docker Deployment (Documentation)

Deployed the trained XGBoost model using FastAPI inside a Docker container to create a portable and consistent prediction service.

Steps Performed:

1. Created a Dockerfile for the FastAPI application.
2. Added all required libraries to requirements.txt.
3. Built the Docker image using the docker build command.
4. Created and started a Docker container using docker run.
5. Verified the running container using docker ps.
6. Tested the deployed API using the FastAPI Swagger interface at `http://127.0.0.1:8000/docs`.





This confirms that:

- ✓ FastAPI is running.
- ✓ The XGBoost model loaded successfully.
- ✓ The scaler is working.
- ✓ The API accepts input correctly.
- ✓ Predictions are returned successfully.

12.Conclusion :

This project successfully developed and deployed an Online Shoppers Purchasing Intention Prediction system using machine learning techniques. After preprocessing the data, applying feature selection, handling class imbalance with SMOTE, and tuning the XGBoost model, the final model achieved strong predictive performance of **88.53%**. The model was successfully deployed using FastAPI and containerized with Docker, providing a scalable and reliable API for real-time purchase prediction.

13. Future Enhancements :

- * Integrate the prediction model with a real-time e-commerce application.
- * Deploy the application on a cloud platform for scalability and availability.
- * Implement automated model retraining and performance monitoring.

Submission Folder Structure

Online_shoppers_purchasing_intention/

|

|— Main Project file.ipynb

|— best_xgb_model.pkl

|— requirements.txt

|— Dockerfile

|— app.py

|— scaler.pkl

Author

Student Name: Yuvalakshmi Karthikeyan

Batch: TN-DS-BN02

Submission Date: 25-07-2026